

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 2 — Misurare e migliorare

Chiusura del modulo

Quiz finale

Tre affermazioni sul Modulo 2: vero o falso? Ditelo a voce

Il Modulo 2, *Prompting avanzato*, si chiude qui: dopo le sue due parti, **progettare il prompt** e **misurare e migliorare**, tre affermazioni per ripassarlo

1. Con i modelli di oggi, «ragiona passo passo» migliora sempre la risposta.
   FALSO — nella registrazione del modulo, il problema degli sportelli (27 persone in fila, 3 sportelli): Haiku, Sonnet e Opus danno **l'orario giusto** di getto e passo passo, e il passo passo allunga la risposta, non la cambia. Il ragionamento c'è già — sui modelli 5.5 non si spegne — e la leva è l'**effort**.

2. Se la risposta rispetta il divieto, il prompt ha funzionato.
   FALSO — nella registrazione, «Non menzionare l'indennizzo» ha **tenuto** in tre prove su tre, su Sonnet. Ma il cliente chiedeva proprio dell'indennizzo, e non ha avuto **risposta**. Si scrive che cosa dire al posto del divieto.

3. «Rispondi solo con un JSON» garantisce il formato.
   FALSO — nella registrazione ha retto su Haiku, Sonnet e Opus, ma una richiesta **non è una garanzia**: la forma la garantisce uno **schema**, in Claude Code `claude -p` con `--json-schema`. E lo schema garantisce la forma, non il contenuto: il telefono del cliente era ancora nel campo.

Da portare a casa

Il Modulo 2 in cinque frasi: progettare il prompt, poi misurarlo

1. Un buon prompt è una **specifica**, non un desiderio: ogni ingrediente che manca — ruolo, contesto, istruzione, formato, vincoli, esempi — è una decisione lasciata al modello
2. Ciò che non cambia da una richiesta all'altra va nel ***system prompt***; il caso e i dati, nel **messaggio**
3. I **tag** separano le istruzioni dai dati; lo **schema** garantisce la forma, non il contenuto
4. Gli **esempi** insegnano la forma; le regole e le etichette si scrivono
5. Un prompt si progetta, poi si **misura**: una variabile alla volta, con i controlli nel codice e un giudice con una rubrica — e il giudice è uno strumento, non l'arbitro

A parità di modello, il prompt è la leva di qualità più potente che avete.

Tutto il modulo in una scheda: `riferimenti/pattern-di-prompt.md`, nel repository delle esercitazioni

Per approfondire

Tutto di Anthropic: documentazione, corsi dell’Academy, video

Documentazione	*Prompting best practices* — il riferimento «vivo»
Documentazione	*Structured outputs*: la risposta vincolata a uno schema
Documentazione	*Run Claude Code programmatically* — `claude -p`, l’uscita JSON e lo schema
Documentazione	*Create custom subagents* — i subagent, come il giudice del laboratorio
Corso Academy	*Claude 101* — 13 lezioni, 2,5 ore
Corso Academy	*AI Fluency: Framework and foundations* — 14 lezioni, 4 ore
Corso Academy	*Claude Code 101* — 12 lezioni, 1,5 ore
Video	*Prompting 101 | Code w/ Claude* — canale Anthropic
Video	*AI prompt engineering: A deep dive* — canale Anthropic
Video	*Prompting for Agents | Code w/ Claude* — canale Anthropic
Video	*The prompting playbook* — canale Claude

Nel prossimo modulo

Modulo 3 — Funzionalità core di Claude (piano Pro)

- In questo modulo il prompt l'avete progettato e misurato quasi sempre in **Claude Code**; nel Modulo 3 entra negli strumenti di **claude.ai**, sul piano Pro
- Nel Modulo 2 claude.ai è passato di sfuggita: la prova guidata con gli esempi, le istruzioni che aggiungete voi, per tutte le chat o per un progetto, la memoria, che può riportare il contesto delle chat passate — nel Modulo 3 è **al centro**
- Il Modulo 3 riparte da qui: la specifica, i tag, gli esempi e la misura li portate con voi — cambia lo strumento, **non il modo di chiedere**

Conoscere che cosa c'è nella cassetta degli attrezzi che avete già pagato.